

UNIVERSIDAD AUTÓNOMA “GABRIEL RENÉ MORENO”

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y
TELECOMUNICACIONES



LAMINAS FRAMEWORK

DOCENTE: Ing. Balcazar Veizaga Evans

MATERIA: Tecnología Web

GRUPO: 17 - SC

INTEGRANTE	REGISTRO
Choque Caceres Pablo Cesar	217011721
Calvimontes Vedia Miguel Angel	217163191
Sandoval Martinez Erick Noel	220036314

Contenido

Antecedentes	4
1.1 ¿Qué es Laminas Framework?	4
1.2 Historia y Transición desde Zend Framework	5
1.3 Objetivos del Proyecto	6
1.3.1 Objetivo general	6
1.3.2 Objetivos específicos	6
2. Arquitectura y características de Laminas Framework	6
2.1 Patrón Arquitectónico MVC	6
2.1.1 Modelo (Model)	7
2.1.2 Vista (View)	7
2.1.3 Controlador (Controller)	8
2.2 Componentes Independientes y Modularidad	8
2.3 Seguridad Integrada	9
2.4 Inyección de Dependencias	9
2.5 Justificación del uso de Laminas Framework frente a otros frameworks PHP	10
3. Stack Tecnológico	11
3.1 Justificación de Fedora Linux	12
3.2 Justificación de PHP 8.5	12
3.3 Justificación de MySQL 8.4	12
3.4 Diagrama de Arquitectura del Stack	13
4. Requisitos del Sistema	13

4.1 Requisitos de Hardware	13
4.2 Requisitos de Software	14
4.3 Extensiones PHP Requeridas	14
4.4 Requisitos funcionales implementados	14
5. Instalación en Fedora Linux (Paso a Paso)	15
5.1 Paso 1: Actualización del Sistema	16
5.2 Paso 2: Instalación de PHP y Extensiones	16
5.3 Paso 3: Instalación de Composer	16
5.4 Paso 4: Creación del Proyecto Laminas.....	17
5.5 Paso 5: Instalación y Configuración de MySQL	17
5.6 Paso 6: Iniciación del Servidor de Desarrollo.....	18
5.6.1 Ejecución mediante Composer.....	18
Conclusión	19
Bibliografía	20

Antecedentes

La Dirección Universitaria de Bienestar Estudiantil y Social, conocida como DUBSS, organiza actividades deportivas universitarias en las que participan diferentes carreras, facultades o equipos representativos de la Universidad Autónoma Gabriel René Moreno. En este tipo de competencias, la planificación de encuentros deportivos requiere registrar equipos, definir horarios, establecer lugares de juego y evitar cruces entre partidos programados.

Tradicionalmente, este proceso puede realizarse de forma manual, mediante hojas de cálculo o registros administrativos básicos. Sin embargo, este método presenta limitaciones importantes, debido a que puede generar duplicidad de datos, errores en la asignación de horarios, cruces entre equipos, pérdida de información y demora en la elaboración del fixture. Además, cuando el número de equipos aumenta, la organización manual se vuelve más compleja y menos eficiente.

Frente a esta necesidad, se plantea el desarrollo de un Sistema Inteligente de Gestión Deportiva, orientado a mejorar la administración de equipos y encuentros deportivos mediante una aplicación web construida con Laminas Framework. El sistema permite centralizar la información en una base de datos MySQL, controlar el acceso mediante autenticación de usuarios y generar encuentros deportivos con apoyo de un módulo externo de inteligencia artificial desarrollado en Python.

Este enfoque permite integrar tecnologías web modernas con un modelo de datos relacional y una arquitectura basada en capas, proporcionando una solución organizada, mantenible y adecuada para un entorno académico e institucional.

1.1 ¿Qué es Laminas Framework?

Laminas Framework es un framework de desarrollo web de código abierto basado en PHP, diseñado para construir aplicaciones web y APIs robustas, escalables y seguras. Se posiciona como la evolución directa del legendario Zend Framework, uno de los frameworks más influyentes en la historia del desarrollo web con PHP. Laminas adopta un enfoque basado en componentes independientes que

permite a los desarrolladores utilizar únicamente las piezas que necesitan, sin la carga de un framework monolítico.

El framework se fundamenta en principios sólidos de ingeniería de software: separación de responsabilidades mediante el patrón MVC (Modelo-Vista-Controlador), inversión de control a través de inyección de dependencias, programación orientada a interfaces, y adhesión a los estándares PSR (PHP Standard Recommendations) definidos por el PHP Framework Interop Group (PHP-FIG). Estos principios garantizan que las aplicaciones construidas con Laminas sean mantenibles, testables y extensibles a largo plazo.

Laminas se distribuye bajo la licencia BSD-3-Clause y es mantenido por la Linux Foundation a través del proyecto Laminas, lo que asegura su continuidad, gobernanza abierta y neutralidad corporativa. A diferencia de otros frameworks PHP que están respaldados por una sola empresa, Laminas cuenta con un modelo de gobernanza comunitaria que involucra a múltiples organizaciones y desarrolladores independientes.

1.2 Historia y Transición desde Zend Framework

El Zend Framework fue creado originalmente por Zend Technologies (ahora parte de Rogue Wave Software y posteriormente Perforce) en 2006 como un framework empresarial para PHP. Durante más de una década, Zend Framework fue el estándar de facto para aplicaciones empresariales PHP, utilizado por organizaciones como la BBC, Cisco y BNP Paribas.

En abril de 2019, Zend Technologies anunció la transición del proyecto hacia una gobernanza comunitaria bajo la Linux Foundation, renombrándolo como Laminas Project. Esta transición se completó en enero de 2020 y representó uno de los movimientos más significativos en el ecosistema PHP: la liberación de un framework empresarial de primer nivel hacia una gobernanza completamente abierta.

La transición no fue meramente cosmética. El equipo de Laminas aprovechó la oportunidad para modernizar la arquitectura, eliminar dependencias obsoletas, mejorar la compatibilidad con versiones recientes de PHP, y reorganizar los componentes bajo el namespace Laminas. Todos los paquetes fueron

migrados de zendframework/* a laminas/* en Packagist, y se proporcionó una herramienta de migración automática para proyectos existentes.

1.3 Objetivos del Proyecto

1.3.1 Objetivo general

Desarrollar un Sistema Inteligente de Gestión Deportiva web, utilizando Laminas Framework, PHP, MySQL y Python, para optimizar la generación del fixture deportivo de la DUBSS UAGRM.

1.3.2 Objetivos específicos

Los objetivos específicos del proyecto son los siguientes:

- Implementar una arquitectura Modelo-Vista-Controlador mediante Laminas Framework, separando la lógica de presentación, control y acceso a datos.
- Desarrollar un módulo de autenticación de usuarios para restringir el acceso al sistema y proteger las funcionalidades administrativas.
- Diseñar una base de datos relacional en MySQL para almacenar usuarios, equipos y encuentros deportivos.
- Mostrar los encuentros generados en un dashboard web mediante vistas html renderizadas por Laminas.
- Aplicar una estructura de carpetas organizada que facilite el mantenimiento, la escalabilidad y la comprensión del proyecto.

Documentar el proceso de instalación, configuración e implementación del sistema sobre Fedora Linux.

2. Arquitectura y características de Laminas Framework

2.1 Patrón Arquitectónico MVC

El patrón Modelo-Vista-Controlador (MVC) es uno de los patrones arquitectónicos más utilizados en el desarrollo de aplicaciones web. Laminas implementa este patrón de manera rigurosa, proporcionando

clases base, interfaces y convenciones que guían al desarrollador hacia una separación limpia de responsabilidades.

En el contexto de Laminas, cada componente del MVC tiene responsabilidades claramente definidas:

2.1.1 Modelo (Model)

El Modelo encapsula la lógica empresarial y el acceso a datos. En el SIGD, los modelos representan las entidades del dominio deportivo: Equipo, Encuentro, Torneo, Jugador y Resultado. Laminas no impone un ORM específico, lo que permite utilizar desde consultas SQL directas con laminas-db hasta ORMs completos como Doctrine. Para este proyecto, se utiliza laminas-db con el patrón Table Gateway, que proporciona un mapeo directo entre tablas de la base de datos y clases PHP.

El patrón Table Gateway encapsula toda la lógica de acceso a una tabla específica en una clase dedicada. Esto permite centralizar las consultas SQL, aplicar validaciones de negocio antes de la persistencia, y facilitar la migración entre motores de base de datos sin afectar la lógica de la aplicación.

2.1.2 Vista (View)

La Vista en Laminas utiliza archivos .phtml (PHP HTML Templates) que combinan marcado HTML con expresiones PHP mínimas para la presentación de datos. Laminas proporciona un sistema de plantillas con soporte para layouts (plantillas maestras), partials (fragmentos reutilizables) y helpers (funciones auxiliares de vista). Los View Helpers son componentes especialmente útiles que encapsulan lógica de presentación recurrente, como la generación de URLs, la renderización de formularios y la gestión de metadatos de página.

El sistema de renderización de Laminas soporta múltiples estrategias de respuesta. Además del renderizado HTML tradicional, es posible configurar estrategias para respuestas JSON (ideal para APIs REST), XML, y formatos personalizados. Esta flexibilidad permite que el mismo controlador sirva tanto páginas web como endpoints de API.

2.1.3 Controlador (Controller)

Los controladores en Laminas heredan de `Laminas\Mvc\Controller\AbstractActionController`. Cada método público que termina en 'Action' (por ejemplo, `indexAction()`, `detalleAction()`) corresponde a una acción que puede ser invocada a través del sistema de enrutamiento. El controlador actúa como orquestador: recibe la solicitud HTTP, extrae parámetros, invoca servicios de negocio, y prepara la respuesta mediante un `ViewModel` que se pasa a la vista.

Laminas también soporta controladores RESTful mediante `AbstractRestfulController`, que mapea automáticamente los métodos HTTP (GET, POST, PUT, DELETE) a métodos del controlador (`getList`, `create`, `update`, `delete`). Esto facilita la construcción de APIs RESTful que siguen las convenciones HTTP estándar.

2.2 Componentes Independientes y Modularidad

Una de las fortalezas más significativas de Laminas es su arquitectura de componentes independientes. A diferencia de frameworks monolíticos donde todo está interconectado, cada componente de Laminas puede instalarse y utilizarse de forma aislada mediante Composer. Esto significa que un proyecto que solo necesita validación puede instalar `laminas-validator` sin cargar el framework MVC completo.

Los componentes principales incluyen: `laminas-mvc` (framework MVC), `laminas-db` (abstracción de base de datos), `laminas-form` (formularios y validación), `laminas-authentication` (autenticación), `laminas-permissions-rbac` (control de acceso basado en roles), `laminas-cache` (sistemas de cache), `laminas-log` (logging), `laminas-mail` (envío de correo), `laminas-session` (gestión de sesiones), `laminas-i18n` (internacionalización), entre muchos otros.

El sistema de módulos de Laminas permite organizar una aplicación en unidades funcionales independientes. Cada módulo contiene su propia configuración, controladores, modelos, vistas y servicios. Esta modularidad facilita la reutilización de código entre proyectos y la separación de equipos de desarrollo por módulos funcionales.

2.3 Seguridad Integrada

Laminas proporciona una suite completa de herramientas de seguridad que protegen las aplicaciones contra las vulnerabilidades más comunes identificadas por OWASP (Open Web Application Security Project):

- Protección contra Cross-Site Scripting (XSS): Los View Helpers de Laminas aplican escape automático de caracteres HTML, previniendo la inyección de scripts maliciosos en las páginas.
- Protección contra Cross-Site Request Forgery (CSRF): El componente laminas-form genera tokens CSRF automáticamente en cada formulario, verificando su validez en cada envío.
- Protección contra SQL Injection: Las consultas parametrizadas de laminas-db y el uso de prepared statements previenen la inyección de código SQL malicioso.
- Validación y Filtrado de Entrada: Los componentes laminas-validator y laminas-filter proporcionan más de 40 validadores predefinidos (email, URL, fecha, rango numérico, expresiones regulares) y filtros de sanitización.
- Gestión Segura de Contraseñas: Laminas utiliza bcrypt por defecto para el hashing de contraseñas, con soporte para ajustar el costo computacional.
- Autenticación y Autorización: laminas-authentication soporta múltiples adaptadores (base de datos, LDAP, HTTP Digest) y laminas-permissions-rbac implementa control de acceso basado en roles.

2.4 Inyección de Dependencias

El contenedor de servicios de Laminas (ServiceManager) implementa el patrón de Inversión de Control (IoC) mediante inyección de dependencias. Este contenedor administra la creación, configuración y ciclo de vida de todos los objetos de la aplicación, asegurando que las dependencias se resuelvan de manera automática y consistente.

La configuración del ServiceManager se realiza declarativamente en archivos de configuración PHP, donde se especifican factories (funciones que crean instancias), aliases (nombres alternativos), invocables (clases sin dependencias), y abstract factories (creación dinámica). Este enfoque declarativo facilita la comprensión de las dependencias del sistema y permite la sustitución de implementaciones para testing.

Un ejemplo práctico en el SIGD: el controlador de equipos depende de un servicio de equipos, que a su vez depende de un repositorio de equipos, que depende de un adaptador de base de datos. El ServiceManager resuelve toda esta cadena automáticamente, creando cada objeto con sus dependencias inyectadas.

2.5 Justificación del uso de Laminas Framework frente a otros frameworks PHP

La selección de Laminas Framework para el desarrollo del Sistema Inteligente de Gestión Deportiva responde a criterios de modularidad, seguridad, mantenibilidad y enfoque empresarial. Aunque existen otros frameworks PHP populares como Laravel y Symfony, Laminas ofrece características especialmente adecuadas para proyectos académicos e institucionales donde se busca demostrar una arquitectura clara y controlada.

En primer lugar, Laminas destaca por su arquitectura basada en componentes independientes. Esto significa que el desarrollador puede instalar y utilizar únicamente los paquetes necesarios para el proyecto, como laminas-mvc, laminas-db, laminas-session, laminas-form o laminas-authentication, evitando cargar funcionalidades innecesarias. Esta característica permite construir aplicaciones más ligeras y adaptadas a los requerimientos específicos del sistema.

A diferencia de Laravel, que proporciona un ecosistema muy completo y altamente integrado desde el inicio, Laminas ofrece mayor libertad arquitectónica. Laravel facilita el desarrollo rápido mediante convenciones y herramientas preconfiguradas; sin embargo, Laminas permite un control más detallado sobre la estructura, las dependencias y la configuración del sistema. Esta característica resulta

útil en un proyecto universitario, porque permite evidenciar con mayor claridad el funcionamiento interno del patrón MVC.

En comparación con Symfony, Laminas comparte un enfoque empresarial y modular, pero mantiene una estructura adecuada para proyectos donde se requiere explicar de manera directa la relación entre controladores, vistas, servicios, rutas y base de datos. Además, Laminas cumple con estándares PSR del ecosistema PHP, lo cual favorece la interoperabilidad con librerías externas y buenas prácticas de desarrollo.

Por estas razones, Laminas Framework resulta apropiado para el desarrollo del SIGD, ya que permite implementar una aplicación web segura, modular, escalable y alineada con principios de ingeniería de software.

3. Stack Tecnológico

El Sistema Inteligente de Gestión Deportiva está construido sobre una pila tecnológica cuidadosamente seleccionada que combina componentes de infraestructura, desarrollo backend-frontend y persistencia de datos. A continuación se detalla cada componente:

Capa	Tecnología	Versión	Tipo	Propósito
Infraestructura	Fedora Linux	44 Workstation	S.O.	Plataforma base de ejecución
Lenguaje	PHP	8.5	Runtime	Lenguaje servidor principal
Framework	Laminas	3.x (MVC Skeleton)	Framework	Arquitectura MVC y componentes
Base de Datos	MySQL	8.4 Community	SGBD	Persistencia relacional
Paquetes	Composer	2.9	Gestor	Gestión de dependencias PHP
Servidor	PHP Dev Server	Integrado	HTTP	Servidor de desarrollo local
Ext. PHP	cli, intl, xml, zip	Nativas	Extensiones	Funcionalidades core de PHP
Ext. PHP	mbstring, mysqlnd	Nativas	Extensiones	Strings multibyte y MySQL

3.1 Justificación de Fedora Linux

Fedora Linux fue seleccionado como sistema operativo por las siguientes razones técnicas:

- proporciona versiones actualizadas de PHP, MySQL y herramientas de desarrollo a través del gestor de paquetes DNF
- es patrocinado por Red Hat y sirve como base para RHEL (Red Hat Enterprise Linux), lo que garantiza estabilidad y soporte a largo plazo
- incluye SELinux para seguridad mejorada del sistema
- tiene ciclos de lanzamiento semestrales que mantienen el ecosistema actualizado
- su gestor de paquetes DNF es rápido, confiable y resuelve dependencias automáticamente.

3.2 Justificación de PHP 8.5

PHP 8.5 fue seleccionado por las mejoras significativas que introduce sobre versiones anteriores: soporte mejorado para tipado estricto y tipos de unión, expresiones match para control de flujo más limpio, atributos nativos (annotations) que reemplazan docblocks, fibras (Fibers) para programación asíncrona, propiedades de solo lectura (readonly properties), enumeraciones nativas (Enums), y mejoras sustanciales de rendimiento respecto a PHP 7.x gracias al compilador JIT.

3.3 Justificación de MySQL 8.4

MySQL 8.4 Community Server fue elegido como sistema de gestión de base de datos relacional por:

- a) soporte completo para transacciones ACID con InnoDB
- b) sistema de claves foráneas para integridad referencial
- c) optimizador de consultas mejorado
- d) window functions y Common Table Expressions (CTEs) para consultas analíticas
- e) soporte JSON nativo para datos semi-estructurados

- f) amplia adopción en la industria y abundante documentación
- g) compatibilidad nativa con PHP mediante la extensión mysqlnd.

3.4 Diagrama de Arquitectura del Stack

La arquitectura del sistema se organiza en cuatro capas principales que interactúan de manera vertical:

Capa	Descripción
Capa de Presentación	Vista (.phtml) renderizada por Laminas View. HTML + CSS + JavaScript servido al navegador del usuario.
Capa de Aplicación	Controladores de Laminas que gestionan las peticiones HTTP, invocan servicios de negocio y preparan ViewModels.
Capa de Lógica de Negocio	Servicios, Entidades y Repositorios PHP que implementan las reglas del dominio deportivo. Incluye el microservicio Python para IA.
Capa de Datos	MySQL 8.4 almacenando equipos, encuentros, resultados y configuración del sistema. Acceso mediante laminas-db.

Cada capa se comunica únicamente con la capa inmediatamente inferior, siguiendo el principio de dependencia unidireccional. La capa de presentación nunca accede directamente a la base de datos; siempre lo hace a través del controlador y los servicios.

4. Requisitos del Sistema

Para ejecutar el Sistema Inteligente de Gestión Deportiva con Laminas Framework en Fedora Linux, se deben cumplir los siguientes requisitos de hardware y software

4.1 Requisitos de Hardware

Componente	Mínimo	Recomendado
Procesador	Dual Core 2.0 GHz	Quad Core 3.0 GHz
Memoria RAM	2 GB	4 GB o superior
Espacio en Disco	5 GB	10 GB o superior
Red	Conexión a Internet	Banda ancha estable

4.2 Requisitos de Software

Software	Versión Mínima	Propósito
Fedora Linux	44 Workstation	Sistema operativo base
PHP	8.1+	Intérprete de lenguaje
Composer	2.0+	Gestor de dependencias
MySQL Server	8.0+	Motor de base de datos
Git	2.30+	Control de versiones
Python	3.10+	Microservicio de IA

4.3 Extensiones PHP Requeridas

Las siguientes extensiones de PHP son necesarias para el correcto funcionamiento de Laminas

Framework y la conexión con MySQL:

Extensión	Paquete DNF	Función
php-cli	php-cli	Interfaz de línea de comandos para ejecutar scripts PHP
php-mbstring	php-mbstring	Soporte para cadenas de caracteres multibyte (UTF-8, Unicode)
php-intl	php-intl	Internacionalización: formatos de fecha, moneda y pluralización
php-xml	php-xml	Procesamiento de documentos XML y DOM
php-zip	php-zip	Manejo de archivos ZIP (requerido por Composer)
php-mysqlnd	php-mysqlnd	Driver nativo de MySQL para PHP (mejor rendimiento que libmysql)

4.4 Requisitos funcionales implementados

Además de los requisitos de hardware y software, el sistema desarrollado contempla un conjunto de requisitos funcionales que representan las principales acciones disponibles dentro de la aplicación web.

El sistema permite el inicio de sesión de usuarios mediante una pantalla de autenticación, donde se validan las credenciales registradas en la base de datos. Una vez autenticado, el usuario con rol de

administrador puede acceder al dashboard principal del sistema, desde el cual se visualizan los encuentros deportivos generados.

También se implementa la gestión de información deportiva mediante una base de datos MySQL, donde se almacenan los equipos participantes y los encuentros programados. Esta información es consultada desde Laminas Framework y presentada mediante vistas dinámicas .phtml.

Otro requisito funcional importante es la generación automática de partidos mediante un script externo desarrollado en Python. Este script se conecta a la base de datos, consulta los equipos registrados, genera emparejamientos deportivos con probabilidades simuladas y guarda los resultados para que posteriormente sean mostrados en el sistema web.

De esta manera, los requisitos funcionales implementados son:

- Inicio de sesión de usuario administrador.
- Protección de rutas internas mediante sesión.
- Visualización de dashboard principal.
- Consulta de equipos registrados.
- Almacenamiento de fixtures en MySQL.
- Visualización de partidos generados desde Laminas.
- Integración entre Laminas Framework, MySQL y Python.

5. Instalación en Fedora Linux (Paso a Paso)

A continuación se describe el procedimiento completo de instalación del entorno de desarrollo. Se utiliza DNF, el gestor de paquetes de Fedora, para levantar un entorno LAMP (Linux + Apache/PHP + MySQL + PHP) moderno.

5.1 Paso 1: Actualización del Sistema

Antes de instalar cualquier paquete, se actualiza el sistema operativo para asegurar que todos los componentes estén en su versión más reciente:

```
$ sudo dnf update -y
```

Este comando descarga e instala las actualizaciones de seguridad, correcciones de errores y nuevas versiones de todos los paquetes instalados en el sistema.

5.2 Paso 2: Instalación de PHP y Extensiones

Se instalan PHP y todas las extensiones requeridas por Laminas Framework con un solo comando:

```
$ sudo dnf install php php-cli php-mbstring php-intl php-xml php-zip php-mysqlnd -y
```

Verificación de la instalación:

```
$ php -v
```

Este comando debe mostrar la versión de PHP instalada (PHP 8.5.x). Adicionalmente, se puede verificar que todas las extensiones están cargadas:

```
$ php -m | grep -E 'intl|mbstring|xml|zip|mysqlnd'
```

5.3 Paso 3: Instalación de Composer

Composer es el gestor de dependencias estándar para PHP. Se instala desde los repositorios de Fedora:

```
$ sudo dnf install composer -y
```

Verificación:

```
$ composer --version
```

Composer permite declarar las librerías de las que depende un proyecto PHP y las administra automáticamente. Es indispensable para Laminas, ya que el framework se distribuye exclusivamente a través de Packagist (el repositorio principal de Composer).

5.4 Paso 4: Creación del Proyecto Laminas

Se crea un nuevo proyecto Laminas utilizando la plantilla oficial MVC Skeleton. Esta plantilla proporciona la estructura de directorios estándar, la configuración inicial y las dependencias base:

```
$ composer create-project laminas/laminas-mvc-skeleton proyecto_grupo17 --ignore-platform-reqs
```

El parámetro `--ignore-platform-reqs` permite la instalación incluso si alguna extensión opcional no está disponible. Durante la instalación, Composer preguntará si se desean instalar componentes opcionales como `laminas-cache`, `laminas-log` y `laminas-serializer`. Para este proyecto se recomienda aceptar todas las opciones por defecto.

Después de la instalación, la estructura del proyecto se genera automáticamente con la organización estándar de Laminas MVC.

5.5 Paso 5: Instalación y Configuración de MySQL

Se instala el servidor MySQL:

```
$ sudo dnf install mysql-server -y
```

Se inicia el servicio y se habilita para inicio automático:

```
$ sudo systemctl start mysqld
```

```
$ sudo systemctl enable mysqld
```

Se ejecuta el script de seguridad para proteger la instalación:

```
$ sudo mysql_secure_installation
```

Durante la ejecución del script de seguridad se recomienda:

- establecer una contraseña fuerte para el usuario root
- seleccionar nivel de validación de contraseña 0 (LOW) para mayor flexibilidad en desarrollo
- deshabilitar el acceso remoto del usuario root

- eliminar usuarios anónimos
- eliminar la base de datos de prueba
- recargar los privilegios.

Es importante resaltar la importancia de `mysql_secure_installation` para proteger los datos del sistema DUBSS (Dirección Universitaria de Bienestar Estudiantil y Social), ya que la base de datos almacenará información sensible de equipos y encuentros deportivos.

5.6 Paso 6: Iniciación del Servidor de Desarrollo

Para probar la instalación, se inicia el servidor de desarrollo integrado de PHP:

```
$ cd proyecto_grupo17
```

```
$ php -S localhost:8080 -t public
```

La aplicación estará disponible en <http://localhost:8080>.

El parámetro `-t public` indica que el directorio raíz del servidor web es la carpeta `public/`, que contiene el archivo `index.php` (front controller) de Laminas.

Al acceder a la URL en el navegador, se debe visualizar la página de bienvenida de Laminas MVC Skeleton, confirmando que la instalación fue exitosa.

5.6.1 Ejecución mediante Composer

Además del servidor integrado de PHP, el proyecto puede ejecutarse mediante el comando configurado en

Composer:

```
composer serve
```

Este comando permite iniciar el servidor de desarrollo definido en el archivo `composer.json`, simplificando la ejecución del proyecto Laminas. Su uso facilita la puesta en marcha de la aplicación sin necesidad de escribir manualmente toda la instrucción del servidor PHP.

Al ejecutar este comando desde la raíz del proyecto, Laminas carga el archivo public/index.php, el cual actúa como Front Controller de la aplicación. Desde este punto se inicializa el framework, se cargan los módulos configurados y se procesan las solicitudes HTTP recibidas desde el navegador.

Conclusión

El desarrollo del Sistema Inteligente de Gestión Deportiva (SIGD) utilizando Laminas Framework ha permitido demostrar la viabilidad y las ventajas de este framework para la construcción de aplicaciones web empresariales en el contexto académico de la Universidad Autónoma Gabriel René Moreno.

A lo largo de este proyecto se han cubierto los aspectos fundamentales del ciclo de desarrollo de software: desde la selección y justificación del stack tecnológico (Fedora Linux, PHP 8.5, Laminas Framework, MySQL 8.4, Composer), pasando por la instalación y configuración del entorno de desarrollo, el diseño del modelo de datos relacional, la implementación de la arquitectura en capas, hasta la integración de un microservicio de inteligencia artificial en Python.

Los principales logros alcanzados incluyen:

- Implementación exitosa de una aplicación MVC completa con Laminas Framework, incluyendo controladores, modelos, vistas y configuración de rutas.
- Diseño de un modelo de datos relacional normalizado con integridad referencial para la gestión de equipos y encuentros deportivos.
- Documentación detallada del proceso de instalación en Fedora Linux, que puede ser reproducido por cualquier estudiante o desarrollador.
- Integración de una arquitectura híbrida con microservicio Python para lógica de inteligencia artificial, demostrando la capacidad de Laminas para interoperar con otros lenguajes y tecnologías.
- Análisis comparativo fundamentado del framework frente a alternativas del mercado, proporcionando criterios de decisión para futuros proyectos.

Las lecciones aprendidas durante el desarrollo incluyen la importancia de: (a) comprender los patrones de diseño antes de usar un framework, (b) configurar adecuadamente la seguridad del entorno desde el inicio, (c) separar responsabilidades para facilitar el mantenimiento y la escalabilidad, y (d) documentar cada decisión técnica para facilitar la transferencia de conocimiento.

Como trabajo futuro, se recomienda: (a) implementar pruebas unitarias y de integración con PHPUnit, (b) agregar autenticación y autorización con laminas-authentication y laminas-permissions-rbac, (c) containerizar la aplicación con Docker para facilitar el despliegue, (d) implementar una API RESTful para consumo desde aplicaciones móviles, y (e) optimizar el algoritmo de programación inteligente con técnicas de machine learning más avanzadas.

En conclusión, Laminas Framework se confirma como una plataforma madura, flexible y robusta para el desarrollo de aplicaciones web en PHP. Su enfoque en componentes independientes, adhesión a estándares y seguridad integrada lo convierten en una opción especialmente adecuada para proyectos que requieren mantenibilidad y escalabilidad a largo plazo. Para estudiantes de Ingeniería en Sistemas, el conocimiento adquirido con Laminas proporciona bases sólidas que son transferibles a cualquier framework o lenguaje de programación.

Bibliografía

- Composer. (s. f.). Composer: A dependency manager for PHP. Recuperado el 7 de mayo de 2026, de <https://getcomposer.org/>
- Fedora Project. (s. f.). Using the DNF software package manager. Fedora Docs. Recuperado el 7 de mayo de 2026, de <https://docs.fedoraproject.org/en-US/quick-docs/dnf/>
- Laminas Project. (s. f.). Laminas MVC documentation: Quick start. Recuperado el 7 de mayo de 2026, de <https://docs.laminas.dev/laminas-mvc/quick-start/>

- Laminas Project. (s. f.). Routing: Laminas MVC documentation. Recuperado el 7 de mayo de 2026, de <https://docs.laminas.dev/laminas-mvc/routing/>
- Laminas Project. (s. f.). Laminas and Mezzio: Enterprise-ready, open-source PHP components and middleware microframework. Recuperado el 7 de mayo de 2026, de <https://getlaminas.org/>
- Linux Foundation. (2019, 17 de abril). The Linux Foundation forms new Laminas project to continue growth and innovation of Zend Framework.
<https://www.linuxfoundation.org/blog/blog/lf-forms-laminas-project>
- Oracle. (s. f.). MySQL 8.4 Reference Manual: FOREIGN KEY constraints. Recuperado el 7 de mayo de 2026, de <https://dev.mysql.com/doc/refman/8.4/en/create-table-foreign-keys.html>
- Oracle. (s. f.). MySQL 8.4 Reference Manual: Introduction to InnoDB. Recuperado el 7 de mayo de 2026, de <https://dev.mysql.com/doc/refman/8.4/en/innodb-introduction.html>
- PHP Group. (s. f.). PDO::prepare. PHP Manual. Recuperado el 7 de mayo de 2026, de <https://www.php.net/manual/en/pdo.prepare.php>
- PHP Group. (s. f.). SQL Injection. PHP Manual. Recuperado el 7 de mayo de 2026, de <https://www.php.net/manual/en/security.database.sql-injection.php>